Retrieval Strategy, Not Just Retrieval: A Controlled Comparison of RAG Pipelines for Detecting Malicious PyPI Packages

Anonymous Author(s)
Submission Id: EASE2025-XXX

Abstract

Large language models (LLMs) can inspect source code for malicious behavior, but they miss subtle attacks such as obfuscated droppers and typosquats whose installation scripts look innocuous. Retrieval-augmented generation (RAG) has been proposed as the remedy, yet prior work couples retrieval to particular knowledge sources and mechanisms, leaving it unclear how much of the benefit comes from retrieval itself and how much from how retrieval is done. We isolate retrieval strategy through a controlled comparison of seven pipelines—a zero-shot baseline and six RAG variants (dense, sparse BM25, hybrid fusion, behavior-summary, contrastive dual-pool, and their combination)—using the same LLM backbone, prompting, and code-only knowledge base throughout, and evaluate on 1,020 test packages. We find that code-level retrieval substantially improves detection, driven by a large recall gain; simple sparse retrieval performs on par with heavy dense retrieval; contrastive retrieval sharply reduces false alarms; and behavior-summary queries are actively harmful. We also analyze the samples that no method detects (metadata-cloned typosquats) and use a static-grounding proxy to measure explanation faithfulness, showing that retrieval reduces unsupported behavior claims.

CCS Concepts

• **Software and its engineering** → **Software development techniques**; • **Security and privacy** → *Systems security*.

Keywords

supply chain, malicious package, PyPI, large language model, RAG, retrieval, typosquatting

ACM Reference Format:

Anonymous Author(s). 2025. Retrieval Strategy, Not Just Retrieval: A Controlled Comparison of RAG Pipelines for Detecting Malicious PyPI Packages. In *Proceedings of Evaluation and Assessment in Software Engineering (EASE 2025).* ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Open-source package registries have become a cornerstone of modern software development, giving developers access to millions of reusable components [5]. The same accessibility, however, makes these ecosystems an attractive target for supply-chain attacks: adversaries inject malicious code into widely used libraries or publish deceptive packages that imitate legitimate ones [7, 8, 13]. Unlike traditional vulnerabilities, which stem from inadvertent errors, malicious packages are deliberately engineered to deceive users, which makes them particularly difficult to detect [9]. Empirical studies show that such packages persist in registries despite existing safeguards [7, 13, 14], and the continued growth of the Python ecosystem keeps expanding the attack surface.

Automated detection of malicious packages has traditionally relied on static and dynamic analysis and on machine-learning classifiers over hand-crafted features [16]. More recently, large language models (LLMs) have been applied to security-oriented code analysis, including malicious package detection [9, 21]. LLMs can reason over source code in natural language and surface subtle indicators of malicious behavior, yet they are also prone to hallucination and to missing attacks whose code looks innocuous [9, 18].

Retrieval-augmented generation (RAG) has been proposed as a general remedy for such LLM shortcomings: before answering, the model conditions on documents retrieved from an external corpus [6, 12]. Within security, RAG has been used to detect vulnerabilities in smart contracts [20], to augment LLM-based vulnerability detection with knowledge derived from CVEs [4], and to support related analyses. The most direct prior study of RAG for malicious PyPI package detection, however, reports a negative result: Ibiyo et al. [9] found that augmenting LLMs with knowledge bases built from YARA rules, GitHub security advisories, and malicious code examples did not reliably improve detection, while few-shot learning was markedly more effective [9]. That study coupled retrieval to specific knowledge sources and a single retrieval mechanism, so it remains unclear whether retrieval is fundamentally unhelpful for this task or whether *how* retrieval is performed—sparse, dense, hybrid, behavior-based, or contrastive—is what determines its value.

In this paper, we address this question through a controlled comparison of seven pipelines that differ only in the retrieval strategy: no retrieval, dense, sparse (BM25), hybrid fusion, behavior-summary, contrastive, and behavior-summary plus contrastive. We use the same LLM backbone, prompting, and code-only knowledge base throughout, and evaluate on 1,020 test packages of a standard malicious/benign PyPI benchmark. Our main findings are:

- Code-level retrieval substantially improves detection over the no-retrieval baseline, driven by a large recall gain.
- Sparse retrieval performs on par with dense retrieval, at a fraction of the infrastructure cost.
- Contrastive retrieval, which shows the model both malicious and benign reference examples, sharply reduces false alarms.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EASE 2025, TBD

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2025/06
https://doi.org/XXXXXXX.XXXXXXX

- Retrieval over AST-derived behavior summaries is lossy and hurts performance, falling below the no-retrieval baseline.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the dataset, the retrieval strategies, the model configuration, and the evaluation metrics. Section 4 reports classification results, explanation faithfulness, and an error analysis. Section 5 interprets the findings and discusses limitations and threats to validity. Section 6 concludes.

2 Related Work

2.1 Malware Detection Techniques

Malware detection has traditionally been approached through static, dynamic, and hybrid analysis [16]. Static analysis examines code without executing it, extracting features such as opcode sequences, control-flow graphs, and file metadata to identify known malicious patterns. Dynamic analysis monitors behavior in controlled environments, capturing runtime information such as API and system-call sequences, network traffic, and file modifications that static analysis may miss, particularly under obfuscation. Hybrid approaches combine both to leverage their complementary strengths. Classical machine-learning classifiers over hand-crafted features—support vector machines, decision trees, and ensemble methods—remain competitive when feature extraction is effective [16]. More recently, LLM-based methods have been applied to analyze code and its descriptive metadata, capturing subtle, context-rich patterns of malicious behavior [9].

In package registries, the same techniques face a distinctive threat model. Duan et al. [5] measured hundreds of malicious packages across PyPI, npm, and RubyGems, showing that most are installed by ordinary users through ordinary `pip install`-style commands, and that name-based imitation (typosquatting) is a dominant attack style. Ohm et al. [14] cataloged attack techniques in open-source software supply chains, including injections that trigger at package-install time through `setup.py` hooks. Empirical studies of the PyPI ecosystem report that malicious packages persist despite registry-level safeguards [7, 13], and graph-based and behavior-modeling approaches have been proposed for detecting them in adjacent ecosystems [8]. Industry efforts such as Datadog's malicious-software-packages dataset [3] have made labeled collections of such packages publicly available, which we use here.

2.2 LLMs for Malicious-Code Detection

Recent work applies LLMs to security classification tasks. Studies on vulnerability reasoning caution that LLMs remain unreliable on subtle security judgments without auxiliary signal [18]. For malicious packages specifically, LLMs have been used to detect malware in the npm ecosystem [21], and Ibiyo et al. [9]—the closest prior work to ours—evaluated LLMs with zero-shot prompting, RAG, and few-shot learning on a dataset of malicious and benign PyPI packages [9]. Its knowledge bases were built from YARA rules, GitHub security advisories, and malicious `setup.py` examples. A central finding of that study was counterintuitive: retrieval augmentation did not reliably help, yielding mediocre accuracy, whereas few-shot learning was markedly more effective. The study, however, coupled retrieval to its specific knowledge sources and a single retrieval mechanism, so it does not separate the effect of *whether* to retrieve from *how* retrieval is done. A complementary line of work applies multi-agent LLM pipelines to package-level evidence such as metadata and behavioral descriptions, reporting improvements from aggregating multiple signals (LAMPS, "Many Hands Make Light Work") [22]. Our work differs in scope and design: instead of changing the knowledge source or inventing new pipelines, we fix the corpus, the model, and the prompt, and vary only the retrieval strategy, which lets us attribute performance differences to retrieval design rather than to data or agent orchestration.

2.3 Retrieval and Retrieval-Augmented Generation

Retrieval-augmented generation combines a retriever with a generator so that the model conditions on relevant external evidence [6, 12]. Dense passage retrieval established embedding-based retrieval as a standard for open-domain question answering [10]; in parallel, classical sparse scoring, notably the Okapi BM25 function, remains a strong lexical baseline with a well-understood probabilistic foundation [15]. Hybrid systems fuse the two, most simply with Reciprocal Rank Fusion [2]. Within security, RAG has been applied to vulnerability detection in smart contracts [20] and to knowledge-level augmentation of LLM vulnerability analysis with CVE-derived documents [4]. For malicious code specifically, retrieving code examples has been the first thing tried [9, 19], yet—to our knowledge—no prior study has systematically compared dense, sparse, hybrid, behavior-based, and contrastive retrieval configurations on the same malicious-package benchmark with the same generator. This is the gap we address.

3 Methodology

3.1 Overview

We study whether, and how, retrieval-augmented generation (RAG) improves an LLM's ability to detect malicious PyPI packages, by comparing seven alternative pipelines E0–E6 that differ only in the *retrieval strategy* they apply before the final classification. At a high level, every pipeline (1) takes the target package's `setup.py` source as input, (2) builds a query representation from it, (3) optionally retrieves reference examples from a knowledge base of previously seen packages, (4) feeds the target together with the retrieved evidence (if any) to an instruction-tuned LLM, and (5) asks the model for a structured verdict: malicious or benign, a confidence score, security-relevant behaviors with source-line citations, and a rationale. Figure 1 illustrates the pipeline.

3.2 Dataset

We reuse the dataset assembled by Ibiyo et al. [9] for the RAG-for-malicious-code study, which itself mines packages from Datadog's malicious-software-packages dataset [3]. The replication package ships pre-split JSON files: a training partition of 1,158 malicious and 3,005 benign packages, and a test partition of 276 malicious and 753 benign packages (1,029 in total). Each entry contains the package name, file list, and the raw `setup.py` content; test entries additionally carry an AST-derived textual behavior description. Nine malicious test entries whose `setup.py` could not be extracted (content string `"Not Available"`) are excluded from evaluation,

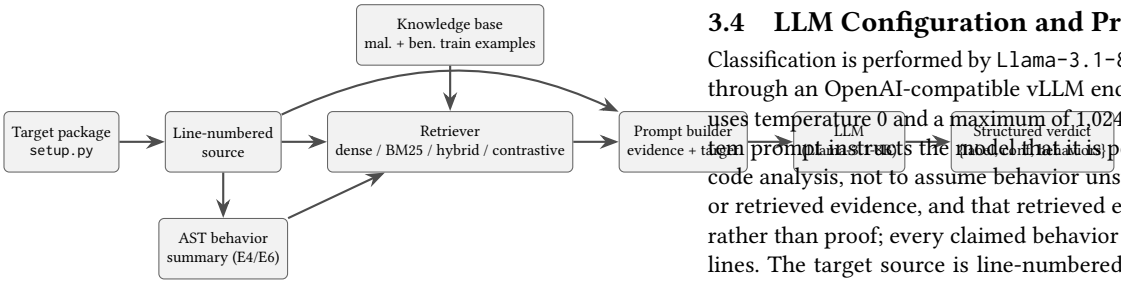

Figure 1: Overview of the evaluated pipelines. All stages are shared across methods except query construction and retrieval (E4/E6 additionally build an AST behavior summary as the query).

leaving 1,020 test samples. The training partition serves as the knowledge base; the test partition is never used for retrieval.

The malicious class is dominated by typosquatted or squatted packages (e.g., `colorama` clones, `requests`-lookalikes) whose installation code frequently hides its malicious behavior in the `setup.py` entry point. This makes the task representative of real-world supply-chain attacks that LLM-based tooling is expected to catch.

3.3 Retrieval Strategies

All methods share the same knowledge base: the raw `setup.py` sources of the 4,163 training packages, indexed in separate malicious and benign pools. We fix the number of retrieved documents to $k = 3$, following the reference design. The seven methods are:

- **E0 — No RAG.** The target source is classified directly, without any retrieved context. This is the controlled baseline.
- **E1 — Dense RAG.** The query is embedded with a sentence embedding model and matched by cosine similarity against the embedded training documents. We use the `Qwen3-Embedding-4B` model [17] and a FAISS `IndexFlatIP` store over L2-normalized vectors [11].
- **E2 — BM25 RAG.** Sparse lexical retrieval with the Okapi BM25 scoring function [15]; no learned components.
- **E3 — Hybrid RAG.** Dense (E1) and sparse (E2) rankings are fused with Reciprocal Rank Fusion [2], $\sum_r 1/(60 + \text{rank}_r(d))$; no fusion weights are trained.
- **E4 — Behavior RAG.** The target is represented by a static behavior summary—imports, suspicious API calls, and behaviors such as shell execution, network access, dynamic execution, or base64 decoding, extracted with Python's `ast` module—and dense retrieval runs over behavior summaries of the malicious pool, analogous to the AST descriptions used in the original dataset.
- **E5 — Contrastive RAG.** Dense retrieval is run twice: top-k documents from the malicious pool *and* top-k from the benign pool, and both groups are shown to the model with explicit `MAL`/`BEN` markers.
- **E6 — Behavior + Contrastive RAG.** Behavior-summary queries (as in E4) combined with dual-pool contrastive evidence (as in E5).

3.4 LLM Configuration and Prompting

Classification is performed by `Llama-3.1-8B-Instruct` [1] served through an OpenAI-compatible vLLM endpoint [11]. Generation uses temperature 0 and a maximum of 1,024 output tokens. The system prompt instructs the model that it is performing static source-code analysis, not to assume behavior unsupported by the source or retrieved evidence, and that retrieved examples are references rather than proof; every claimed behavior must cite target source lines. The target source is line-numbered before it is embedded in the prompt. The output schema is enforced as JSON and requires `verdict` (malicious/benign), `confidence`, `behaviors` (a list of {type, evidence} objects) and `rationale`. Retrieved documents are labeled with stable identifiers (`MAL_*` / `BEN_*`) and their retrieval scores.

3.5 Evaluation Metrics

Classification. We report precision, recall, and F1 for the malicious class, benign-class F1, macro-F1, balanced accuracy, false-positive rate (FPR), and false-negative rate (FNR). Because JSON decoding can fail (e.g., output truncation by the serving backend), we additionally report *coverage*: the fraction of samples whose response parsed successfully; unparsed samples are counted as failures and excluded from metric denominators, so that success and failure modes are reported separately.

Explanation faithfulness. As an approximation of whether the model "hallucinates" security-relevant behaviors, we ground each claimed behavior in a lightweight static checker built on Python's `ast` module, which flags shell execution, process creation, network access, file writes/deletion, dynamic execution, environment access, base64 decoding, remote download, and persistence. Free-form behavior names produced by the model are keyword-normalized to this vocabulary; a claim is *supported* when the corresponding static flag is present on any line of the target. We report the unsupported-claim rate = unsupported/claimed, behavior precision and behavior recall. Because static analysis is neither sound nor complete on obfuscated code, we treat these figures as an *automated static-grounding proxy*, not as hallucination ground truth.

3.6 Implementation and Reproducibility

The pipeline is implemented in Python 3.12 as a modular package (retrieval, static analysis, prompting, evaluation modules) without framework dependencies; BM25 uses `rank-bm25`, dense indexes use `faiss`, and LLM/embedding calls go through a single OpenAI-compatible client so that any endpoint can be swapped by environment variables. All LLM responses, embeddings, and retrieval results are cached (SQLite) keyed by prompt hash, sample content, evidence IDs, and generation parameters; retrieval indexes are persisted on disk; and every run records its configuration, seed, model IDs, and git commit. Run scripts for all seven methods, the evaluation code, and raw per-sample outputs are included in the replication package.

4 Results

4.1 Experimental Setup

We evaluate seven methods (E0–E6, Section 3.3) on the 1,020 usable test samples (267 malicious, 753 benign), all served by the same `Llama-3.1-8B-Instruct` model with identical prompting, decoding parameters ($T = 0$, 1,024 max tokens), and $k = 3$ retrieval settings, so that the only varying factor is the retrieval strategy. Dense indexes use `Qwen3-Embedding-4B`; BM25 uses Okapi BM25 with a standard implementation. The nine samples with unavailable source are excluded from every method.

4.2 Main Results

Table 1 reports classification performance. Three findings stand out.

Table 1: Classification results on the benchmark test set (1,020 samples; 267 malicious). Coverage is the fraction of samples whose structured response parsed successfully; metrics are computed over parsed samples. Best value per column in bold.

Method	Mal. P	Mal. R	F1	Ben. F1	BAcc	FPR	Cov.
E0 No RAG	**1.000**	0.798	0.888	0.973	0.899	**0.0%**	0.925
E1 Dense	0.972	**0.972**	**0.972**	**0.991**	**0.981**	0.9%	0.975
E2 BM25	0.964	**0.972**	0.968	0.989	0.980	1.2%	0.975
E3 Hybrid	0.976	0.964	0.970	0.990	0.978	0.8%	0.976
E4 Behavior	**1.000**	0.782	0.878	0.964	0.891	**0.0%**	0.977
E5 Contrastive	0.996	0.948	0.971	0.990	0.973	**0.1%**	0.927
E6 Beh.+Contr.	**1.000**	0.788	0.881	0.966	0.894	**0.0%**	0.974

Finding 1: retrieval improves detection. The zero-shot baseline (E0) achieves F1 0.888 with perfect precision but misses roughly one in five malicious samples (recall 0.798). Dense retrieval (E1) lifts malicious recall to 0.972 and F1 to 0.972 (+8.4 points) with a balanced accuracy of 0.981; BM25 (E2) and hybrid (E3) essentially match this level (F1 0.968–0.970). Retrieval is thus useful for malicious-code detection, and the gain comes almost entirely from exposing the model to similar *code*, regardless of the similarity mechanism.

Finding 2: sparse retrieval is not worse than dense retrieval. BM25 (0.968) is within 0.4 F1 points of dense (0.972) at a fraction of the engineering cost—an in-memory term index, no embedding model, no vector store. Fusing both (E3) does not beat the better of the two, consistent with prior RAG literature on short, keyword-heavy security queries. For this task, lexical similarity over code tokens is a strong and cheap baseline that future work should not skip.

Finding 3: contrastive evidence trades recall for the fewest false alarms. E5, which shows the model both malicious and benign reference groups, yields the best precision among RAG methods (0.996) with FPR of 0.1%—an order of magnitude below dense/BM25/hybrid (0.8–1.2%)—at the cost of recalling slightly fewer malicious samples (0.948 vs. 0.972). In a setting where triaging false alarms dominates cost, E5 dominates.

Finding 4: static behavior summaries are lossy. Replacing the query with an AST-derived behavior summary (E4) or adding it on top of contrastive retrieval (E6) actively hurts: F1 drops to 0.878/0.881, below the no-RAG baseline in recall-adjusted terms, although all misclassifications remain false negatives and FPR stays at zero. The summaries discard identifier names, call structure and file-path information that the model uses to recognize attacks such as typosquats; retrieving behavior-summary neighbors does not compensate.

4.3 Faithfulness of Explanations

Table 2 reports the static-grounding proxy computed by keyword-normalizing the model's free-form behavior claims to our static vocabulary and checking each against `ast`-derived behavior flags. Unsupported-claim rates are high across the board (0.59–0.72), a caveat we discuss in Section 5, but the ranking is informative: the no-RAG baseline is the least grounded (0.717), all retrieval variants reduce unsupported claims, and behavior-based queries (E4) yield the lowest rate (0.593) with the best claim precision pair (E4/E5: 0.387/0.421). Retrieving explicit evidence therefore correlates with more grounded, less extrapolated explanations, and behavior prompts in particular restrain the model to security-relevant vocabulary.

Table 2: Automated static-grounding statistics for model-claimed behaviors (security-relevant, normalized claims only). `Unsup.` = unsupported claimed behaviors / all claimed; `Beh. P` = supported claims / claimed; `Beh. R` = claimed-and-supported behaviors / statically observed behaviors.

Method	Claims	Unsup.	Beh. P	Beh. R
E0 No RAG	427	0.717	0.282	0.427
E1 Dense	560	0.657	0.312	0.521
E2 BM25	596	0.685	0.294	0.517
E3 Hybrid	568	0.665	0.314	0.525
E4 Behavior	526	**0.593**	0.387	**0.531**
E5 Contrastive	455	0.615	**0.421**	0.457
E6 Beh.+Contr.	697	0.694	0.289	**0.543**

4.4 Error Analysis

Consistently missed samples. Five malicious packages are misclassified as benign by *all seven* methods: `parseweb-v3`, `kazer12-v2`, and `testdufou-v2` are near-verbatim metadata clones of the legitimate `colorama` package (same authors, description and project URL in `setup.py`) that differ only by name; `pepequests-v0.0.1` is a minimal typosquat of `requests` whose `setup.py` contains no executable behavior at all; and `ucap-v3.6.1` hides its malicious setup behind innocuous packaging boilerplate. One further typosquat, `simplyjson`, is missed by six of seven methods. These failures are structural: when the *specific file we analyze* carries no distinguishing signal, no retrieval over the same file type can help, and name-level analysis (e.g., similarity to popular packages) would be required.

Recurring false positives. Two benign packages are flagged as malicious by three of seven methods (`authcaptureproxy-1.1.4`,

`undetected-chromedriver-3.1.5.post4`), both of which legitimately exercise capture-proxy and privilege-heavy browser automation—code patterns orthographically similar to credential-theft trojans in the knowledge base. Spot checks of the sampled false positives suggest they arise from the model over-extending retrieved malicious evidence, the failure mode that contrastive retrieval (E5) is designed to suppress: indeed E5 flags neither of these two packages.

Parse failures. Structured-output decoding fails on 2.3–7.3% of samples depending on the method. The no-RAG baseline (6.5%) and contrastive method (7.3%) are worst; both correspond to the largest average prompt complexity (long obfuscated targets for E0; dual evidence blocks for E5), consistent with backend truncation of long outputs. We treat coverage as a first-class result rather than silently dropping failed samples.

5 Discussion

5.1 Interpretation of Results

Reconciling with the negative result of prior work. The closest prior study reported that RAG did not reliably help LLM-based detection and that few-shot learning was more effective [9]. Our findings complement, rather than contradict, that result: we show that retrieval *can* deliver large gains when the knowledge base is code-only and retrieval is matched to the classification granularity (per-file, $k = 3$, raw-code queries). The prior study's knowledge sources (YARA rules, security advisories) describe packages at a different abstraction level than the target `setup.py`, and its single dense mechanism left no way to separate retrieval design from retrieval presence. The implication for practice is that "RAG" is not a single intervention: both the corpus and the retrieval strategy need to be chosen deliberately, and our comparison provides a first controlled map of that design space.

Retrieval presence matters more than retrieval mechanism. Across dense, sparse, and hybrid variants, malicious recall settles around 0.96–0.97 versus 0.80 with no retrieval—the decisive factor is that the model sees similar code at all, not which similarity function found it. We attribute this to the structure of the problem: passages of malicious source are frequently near-duplicates (copy-pasted droppers, shared obfuscation snippets), which both lexical and semantic similarity recover equally well.

Sparse retrieval is a strong, low-cost baseline. BM25 has no embedding model, no GPU, and no index-approximation tuning, yet yields 0.968 F1 vs. 0.972 for a 4B embedding model. For replication-friendly pipelines, we recommend starting with BM25 and upgrading only if a failure analysis shows semantically-paraphrased attacks slipping through.

Contrastive retrieval offers a favorable precision/recall trade-off. Retrieving malicious examples *only* biases the model toward positives; adding retrieved benign look-alikes lets it weigh counterevidence. The 0.1% FPR of E5 is attractive when analysts must triage alarms, and the mechanism is orthogonal to retriever choice—a practitioner may obtain the same effect with BM25 over both pools.

Behavior summaries reduce the signal. The AST-based representations tested here (E4, E6) discard too much (identifiers, URL structure, file names, control flow). Their recall regresses to no-RAG levels while their recall-adjusted F1 falls below the baseline, which we interpret as evidence that queries should stay close to raw code whenever the generator can accept it.

5.2 Limitations

Our work has several limitations:

- **Single model and single dataset.** All findings are for one generator (`Llama-3.1-8B-Instruct`) and one benchmark; larger or stronger models may compress the gaps between retrieval strategies.
- **Single file granularity.** Analysis is limited to `setup.py`. Malware hidden in auxiliary files is invisible to every method tested, which partly explains the samples no method detects.
- **Fixed retrieval settings.** We fix $k = 3$ (per the reference design); a full $k \in \{1, 3, 5\}$ sweep and corpus-side ablations (e.g., leaving retrieved documents unlabeled) remain for future work.
- **AST-based query extraction is unrefined.** Our behavior summaries are simple import/call inventories; a richer, security-focused static analyzer might restore some of the lost signal.

5.3 Threats to Validity

Internal. Prompt wording is confounded with method (evidence blocks differ). We mitigated this through one shared system prompt, common output schema, stable document identifiers, and identical decoding parameters. Faithfulness scoring is an automated proxy: keyword normalization and AST detection are neither sound nor complete, so grounding numbers should be read comparatively, not absolutely. Parse failures were excluded from metric denominators and reported as coverage to avoid silently dropping or crediting failed samples.

Construct. "Ground-truth" labels originate from the reference dataset; samples whose code could not be extracted (9/1029) were removed, which slightly privileges clean verifiable cases. Classification error is kept distinct from explanatory hallucination: we score the verdict against the label and score the explanation separately against static evidence.

External. The malicious population is largely typosquats from a specific observation period; findings may not transfer to other attack families, registries, or later temporal windows. We therefore treat the study as a controlled comparison rather than an absolute capability claim.

5.4 Future Work

Three directions follow directly. First, replicate over a model panel (2–4 generators) and a second dataset (e.g., `LAMPS` D1) to test whether strategy rankings generalize. Second, extend detection beyond `setup.py`: to whole-package multi-file evidence and, for the typosquat class that defeats every method here, to package-name

similarity features. Third, bring the evidence pipeline closer to production: cross-dataset and temporal splits of the knowledge base, higher k with LLM-based relevance filtering cascades in the spirit of CRAG [19], and a deployed triage evaluation weighting FPR over retrieval efficiency.

5.5 Data and Code Availability

Raw per-sample outputs for all seven methods, the evaluation scripts, and run scripts are provided as a replication package alongside this report; the underlying dataset is the replication package of Ibiyo et al. [9].

6 Conclusion

This paper asked what retrieval strategy, if any, best supports LLM-based detection of malicious PyPI packages. Using the dataset of Ibiyo et al. [9] and a fixed `Llama-3.1-8B-Instruct` generator, and a code-only knowledge base, we compared a zero-shot baseline against dense, sparse, hybrid, behavior-based, and contrastive retrieval.

Three conclusions appear robust. Any code-level retrieval is a large win, lifting malicious-class F1 from 0.888 to 0.97, mostly through recall. The mechanism matters surprisingly little: plain BM25 matches a 4B embedding retriever within 0.4 F1 points, so sparse retrieval is a strong, low-cost default. And contrastive retrieval offers a favorable precision/recall trade-off: retrieving benign counterexamples cuts the false-positive rate to 0.1% at 0.948 recall. Conversely, reducing queries to AST behavior summaries is counterproductive (F1 0.878), demonstrating that signal loss in query construction, not retrieval per se, is where methods fail.

We also documented the residual failures that no retrieval over `setup.py` can fix—metadata-cloned typosquats—and the recurring false positives over security-adjacent legitimate packages, as guidance for the next generation of designs, which should extend evidence beyond a single file, add name-level signals, and calibrate retrieval toward triage workflows.

References

[1] AI@Meta. 2024. The Llama 3 Herd of Models. https://arxiv.org/abs/2407.21783.

[2] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*. 758–759.

[3] Datadog Security Labs. 2023. malicious-software-packages-dataset. https://github.com/datadog/malicious-software-packages-dataset.

[4] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. 2024. Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG. https://arxiv.org/abs/2406.11147.

[5] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Network and Distributed Systems Security (NDSS) Symposium*.

[6] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. https://arxiv.org/abs/2312.10997.

[7] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. 2023. An Empirical Study of Malicious Code In PyPI Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 166–177.

[8] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 1146–1158.

[9] Motunrayo Ibiyo, Thinakone Louangdy, Phuong T. Nguyen, Claudio Di Sipio, and Davide Di Ruscio. 2025. Detecting Malicious Source Code in PyPI Packages with LLMs: Does RAG Come in Handy?. https://arxiv.org/pdf/2504.13769. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*.

[10] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781.

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[13] Genpei Liang, Xiangyu Zhou, Qingyu Wang, Yutong Du, and Cheng Huang. 2021. Malicious Packages Lurking in User-Friendly Python Package Index. In *Proceedings of the IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 606–613.

[14] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, 23–43.

[15] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.

[16] Jagsir Singh and Jaswinder Singh. 2021. A Survey on Machine Learning-Based Malware Detection in Executable Files. *Journal of Systems Architecture* 112 (2021), 101861.

[17] Qwen Team. 2025. Qwen3 Embedding Series. https://arxiv.org/abs/2505.13878.

[18] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. 2024. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks. In *IEEE Symposium on Security and Privacy (S&P)*.

[19] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. https://arxiv.org/abs/2401.15884. In *Findings of the Association for Computational Linguistics: NAACL*.

[20] Jeffy Yu. 2024. Retrieval Augmented Generation Integrated Large Language Models in Smart Contract Vulnerability Detection. https://arxiv.org/abs/2407.14838.

[21] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. 2024. Shifting the Lens: Detecting Malware in npm Ecosystem with Large Language Models. https://arxiv.org/abs/2403.12196.

[22] Muhammad Umar Zeshan, Motunrayo Ibiyo, Claudio Di Sipio, Phuong T. Nguyen, and Davide Di Ruscio. 2026. Many Hands Make Light Work: An LLM-based Multi-Agent System for Detecting Malicious PyPI Packages. https://arxiv.org/abs/2601.12148. *Journal of Systems and Software* (2026).